Examen PAA

Coș

Grafuri

PRIM ← o singură comp. conexă
pornim de la x
NEORIENTAT

KRUSKAL ← mai multe comp. conexe
NEORIENTAT

} Arbore de acoperire minim

DIJKSTRA ← drum minim
de la A la B
ORIENTAT & NEORIENTAT
muchii > 0

PARCURGERI ← cuprindere - drum cel mai scurt fără cost
adâncime ← comp. conexe
dacă ∃ drum

Arbori

Arbori binari

```
typedef struct Nod
{ int cheie;
  struct Nod *st, *dr;
} Nod;
```

- nod_nou
- inserare
- căutare
- min_subarbore - folosit la ștergere
- ștergere ← 1 fiu ← st.
  dr.
  2 fii - cu succ. min
- inordine
- golire

*folosit când se cere cod unic sau ceva sortat crescător sau descrescător

Scanned with CamScanner

Arbori regăsire

```
typedef struct Trie
{ struct Trie* copii [ALF];
  int final;
};
```

- nod_nou
- inserare
- cauta
- sterge
- afisare – extra la Recuput
- golire

} apropape la fel

* Șiruri de caract.
* se referă la prefixe
* contează cât de lung e cuv., nu câte sunt

Examen PAA

TEORIE

Arbori generalizați & arbori binari

1. Nodurile se prelucrează la traversarea în:
   - → preordine - prima oară
   - → inordine - a doua oară
   - → post ordine - ultima / a treia oară

2. Fie X nod în arbore binar
   X are doi copii
   Y - nod terminal succesor X parc. în inordine
   => Y - nu are copil în dreapta.

3. X nod arbore binar
   X are copil în dr.
   Y - succ. lui X parc. în ordine, nu are copii
   => nicio afirmație nu e corectă

4. X nod arbore binar
   X - are doi copii
   Y - succ. lui X inordine
   => Y nu are copil în stânga

5. nr. max. noduri arbore binar înălțime h => $2^h - 1$
6. nr. max. noduri nivel i arbore binar => 2^{i-1}
7. nod oarecare arbore binar → are cel mult doi fii
8. nr. noduri frunză arbore binar complet => $\frac{n+1}{2}$ $(n+1)/2$
9. Toți subarborii unui arbore optim sunt optimi
10. orice structură de arbore gen. poate fi trans. în arbore binar

Scanned with CamScanner

AVL

1. căutare cheie — AVL $O(\log_2 N)$
   * cel mai rău caz. — binar ordonat nechilibrat $O(N)$
2. În AVL frunzele nu se găsesc la același nivel
3. AVL sunt la fel ca binari ordonați (adâncime) simpli mai greoi la inserție și ștergerea cheii.
4. dif. max. de nivel între nodurile frunză
   $\Rightarrow \log(n)$
* 5. Procesul de ștergere nu se oprește după echilibrarea primului nod dezechilibrat întâlnit

Coduri Huffman

1. se codifică fiecare caract. printr-o sec. de 0, 1 a.î. codul unui caract. să nu fie PREFIX pt. altul
2. un cod prefix poate fi reprez. printr-un arbore binar, dacă nodurilor TERMINALE li se atribuie litere, iar ramurilor 0/1.
3. Toți subarborii unui arbore binar optim sunt optimi.
4. Arbore optim → nodurile sortate descr. — greutate mare
   → nodurile sortate crsc. — greutăți mici
5. Arbore binar optim - arbore a cărui structură conduce la un cost minim.
6. Afirmații adev.
   → folosit la compresie
   → niciun cod nu e prefixul altuia
   → necesită cunoașterea frec. pt. fiecare simbol
* 7. deplasare: 0 stânga
   1 dreapta

1.

2.

Arbori B

→ noduri - pag.
→ perfect echilibrați
→ fiecare pag. are între $N-2N$ val.
→ toate pag. terminale pe același nivel

1. alegem arbore B în loc de AVL pt. indexare bază de date pt. că datele se preiau din mem. externă în blocuri
2. cu cât e mai mare ordinul N => nr. scindărilor de pag. e mai mic
3. drumuri rădăcină - oricare nod terminal => egal
4. își reduce din dimensiune dacă - răd. e redusă la dim. zero.
5. inserție nod în pag. plină => scindare
6. utilizați pt. gestionarea unor date aflate în mem externă
7. crește în înălțime când are loc scindarea pag. răd.
8. echilibrat din pct. de vedere al înălțimii
9. AVL necesită mai multe accesări decât B.
10. inserția unei chei într-un arbore B se soldează inserând cheia corsp. la locul potrivit în sec... FALS

GRAFURI

1. ordin graf = nr. noduri
   grad graf = grad max. nod
2. nr. arce graf complet
   - neorientat: $\frac{N(N-1)}{2}$
   - orientat: $N(N-1)$
3. structură eficientă - traversare suprindere: coada
4. graf conex aciclic = arbore
5. nr. arce graf
   - neorientat aciclic: $N-1$
   - conex aciclic: $N-1$
   - orientat aciclic: $\frac{N(N-1)}{2}$

Scanned with CamScanner

6. graf biconex [conține cel puțin un ciclu
nu are pct. de articulație
7. arborele de acoperire minim NU e unic
8. traversare graf - reținem nodurile vizitate
9. n comp. conexe => n-1 muchii pt. a le uni
10. pct. de articulație → dacă îl ștergem rupem graful în mai multe comp. conexe
11. nr. max. arce graf bipartit
- împărțim N în două mul. admise și care înmulțite dau mai mult ca alte perechi.
12. nr. de arce influențează timpul de parcurgere a unui graf.

ARBORI DE REGĂSIRE

→ cheile nu sunt reținute în noduri
→ rădăcina asociată șirului nul
→ un prefix comun
→ grad maxim 26 (nr. de litere)
→ timp căutare O(N), N - lung. cheii
→ timp execuție nu depinde de nr. de înregistrări.
→ consum de memorie

Extra:
→ Un nod A se numește predecesor / strămoș al nodului B dacă ∃ drum de la A la B.

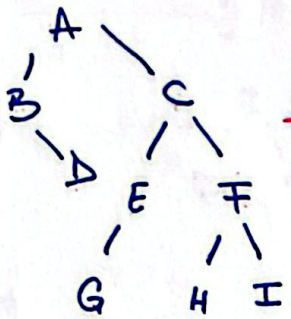
Scanned with CamScanner

Examen PAA

Exercitii

Arbori generalizati + binari

Parcurgeri



preordine RSD

A B D C E G F H I

inordine SRD

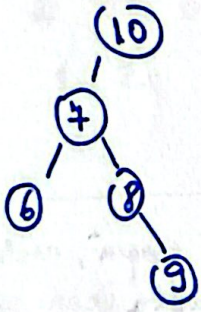
B D A G E C H F I

postordine SDR

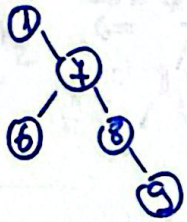
D B G E H I F C A

Arbori binari

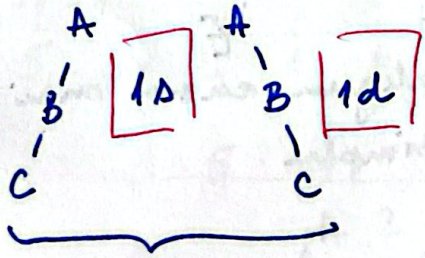
10, 7, 6, 8, 9



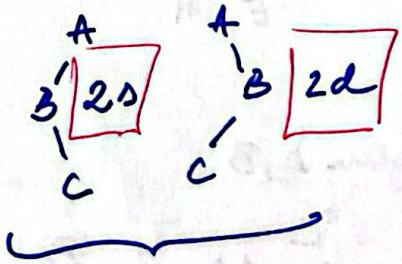
1, 7, 6, 8, 9



AVL

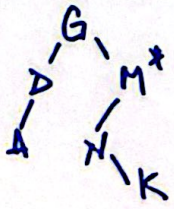


B vine sus



C vine sus

* exemplu.

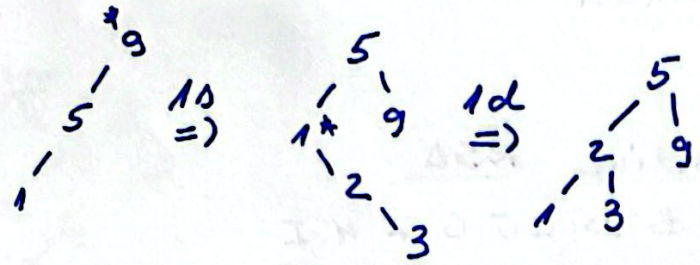


=> 2 s.

Scanned with CamScanner

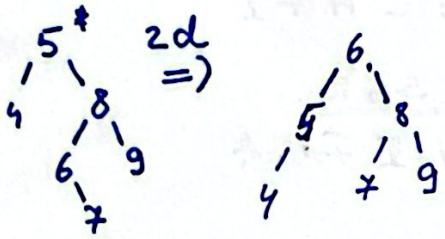
Config. AVL

1. 9 5 1 2 3

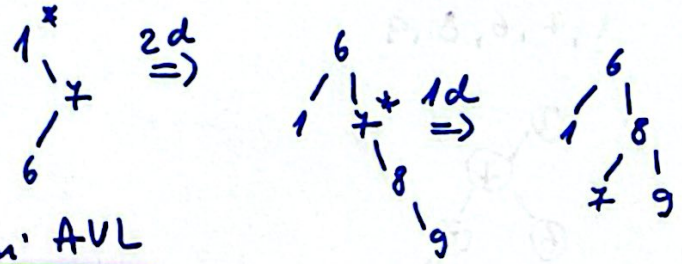


* caut dez. de jos în sus

2. 5 4 8 6 9 7

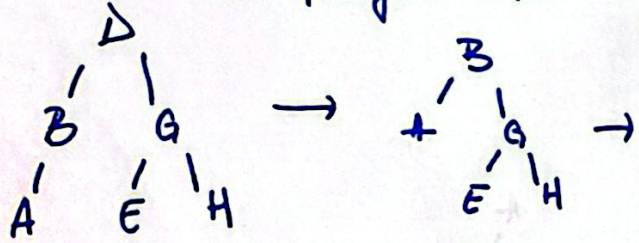


3. 1 7 6 8 9

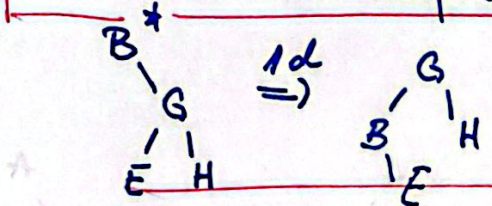


Stergeri AVL

1. Ștergem D, A

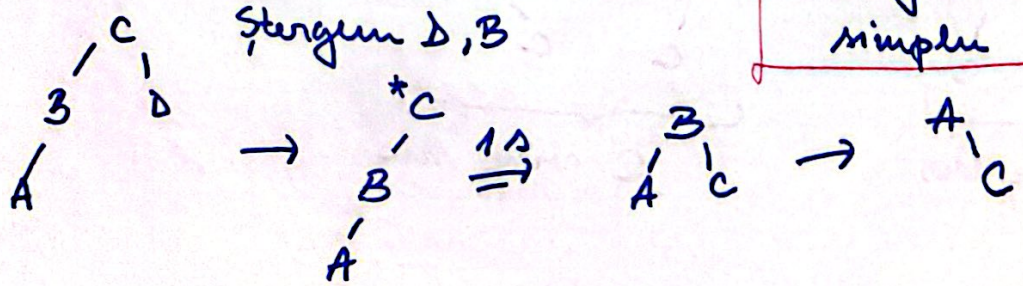


* dacă nodul are copii, vrem cel mai din dreapta care e în stânga



↑ alegem cazul mai simplu

2. Ștergem D, B



Scanned with CamScanner

Coduri Huffmann

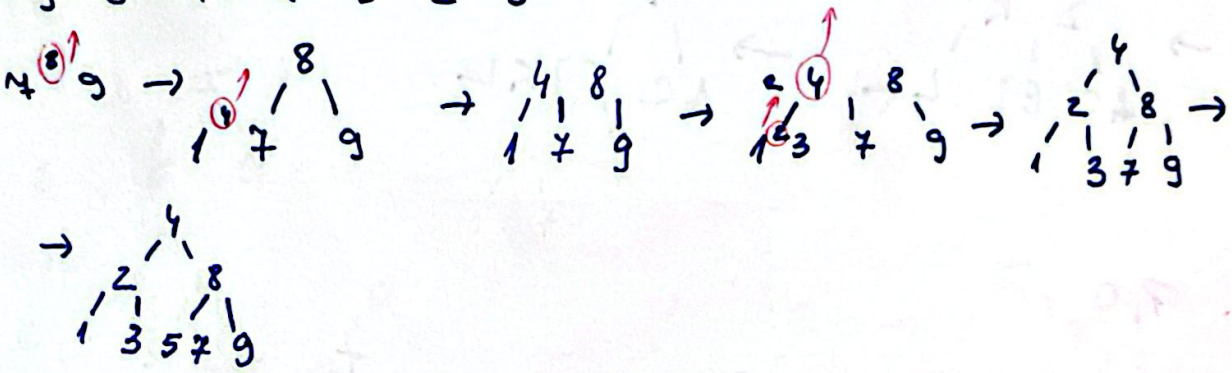
* vezi foaie test.

Arbori B

Adăugări

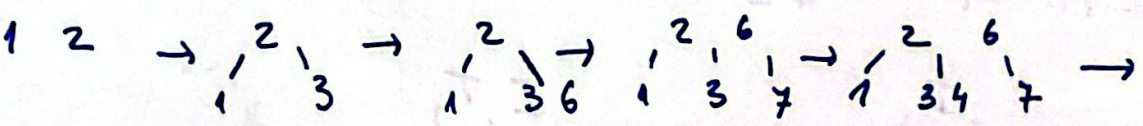
1. N = 1

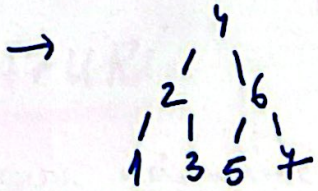
7 9 8 1 4 3 2 5



2. N = 1

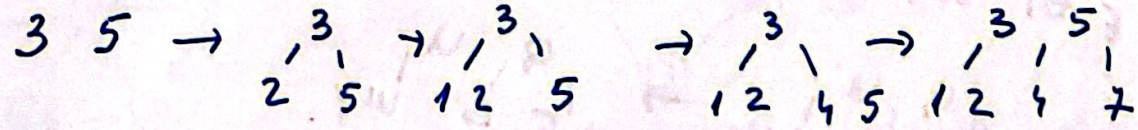
2 1 3 6 7 4 5





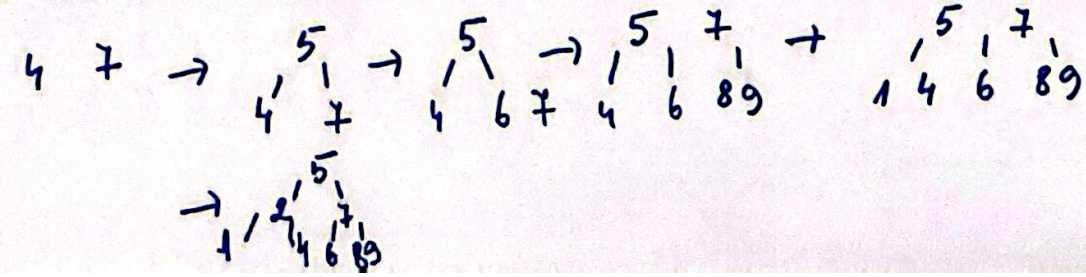
3. N = 1

5 3 2 1 4 7



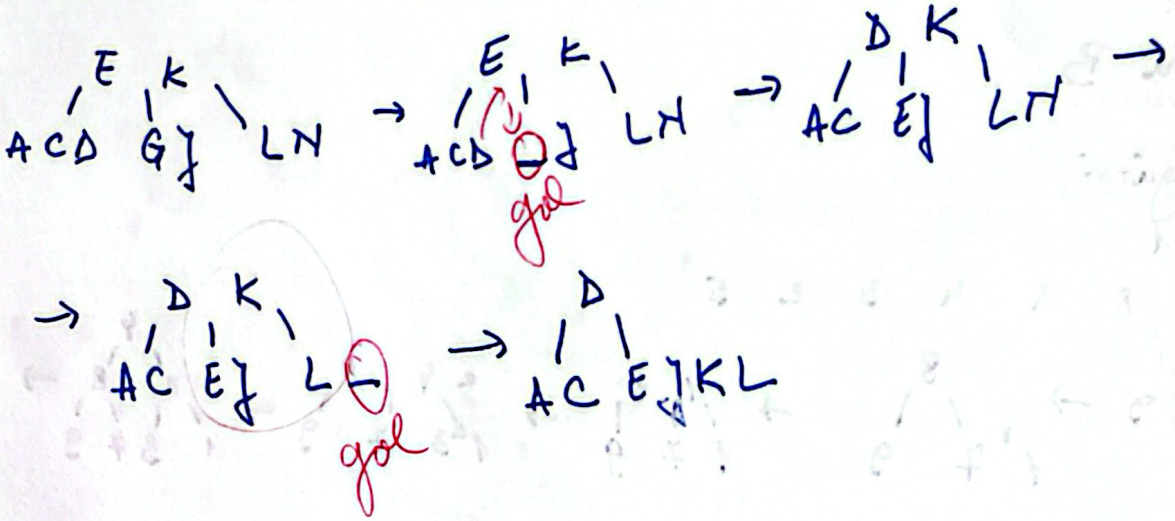
4. N = 1

7 4 5 6 9 8 1 2

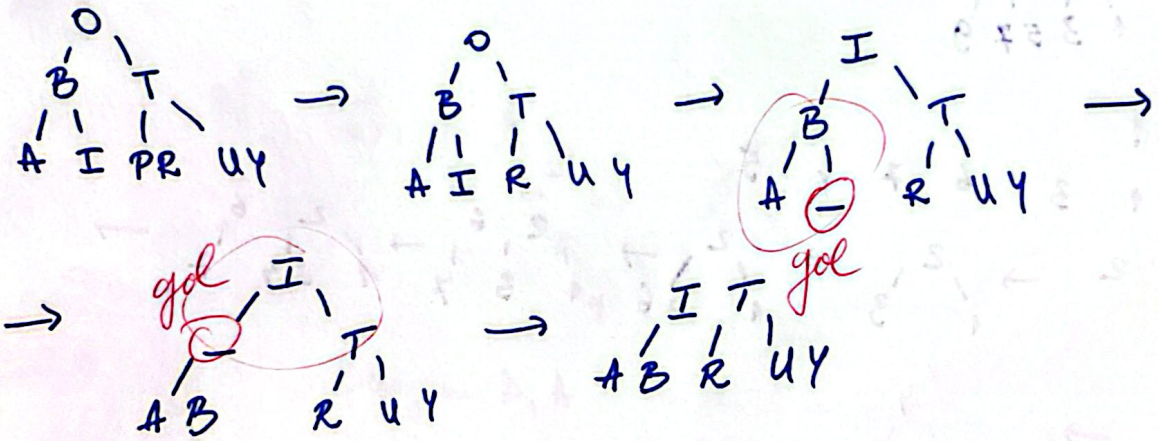


Stergere

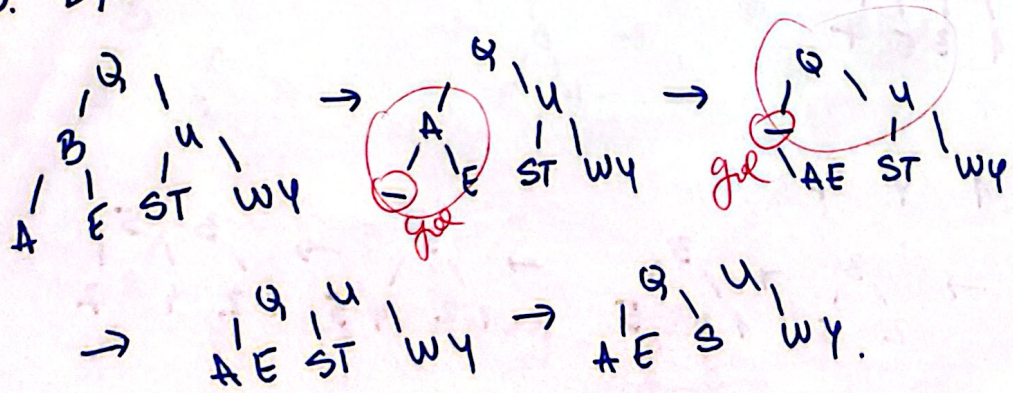
1. N=2 G, N



2. P, O

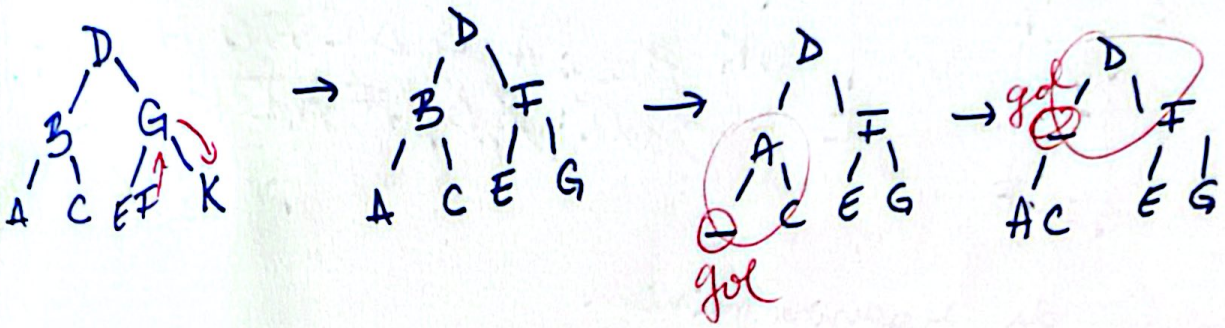


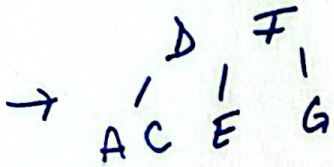
3. B, T



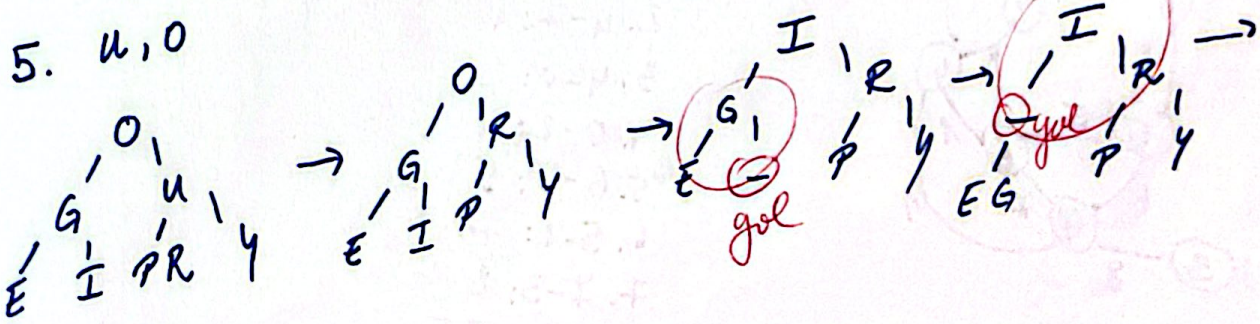
Scanned with CamScanner

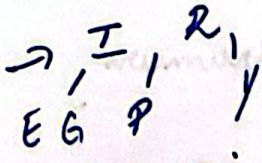
1. K, B





5. U, O

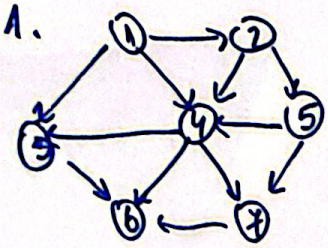




GRAFURI

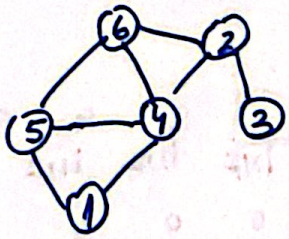
Parcurgere adâncime

1.



de la 1

1 → 2 → 4 → 3 → 6 → 7 → 5



de la 2

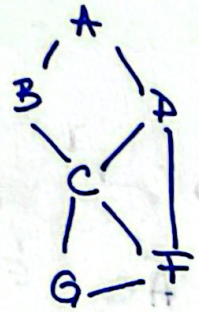
2 → 3 → 4 → 1 → 5 → 6

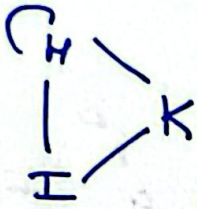
Parcurgere cuprindere

Parc.:	1	2	3	4	5	6	7
Coadă:	1	2	4	3	6	7	5

Scanned with CamScanner

Parc. cuprindere.



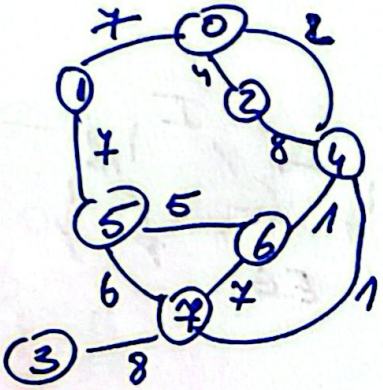


de la index cel mai mic din fiecare comp.

Parc: A B D C F G H I K.

Coadă: ~~B D C F G I K~~

Arbore de acoperire minim

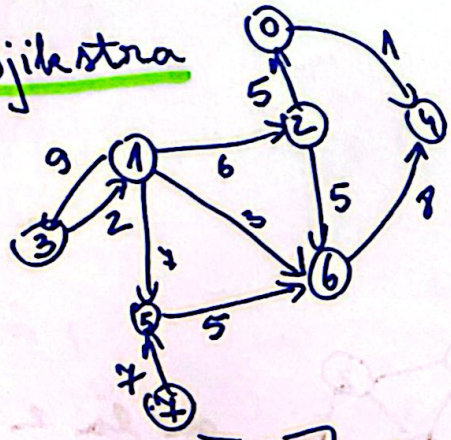


m-1 muchii

=> 7. 1. 4-6 : 1
2. 4-7 : 1
3. 4-0 : 2
4. 0-2 : 4
5. 6-5 : 5
6. 5-1 : 7
7. 7-3 : 8

28

Djikstra



1. lung. minimă drumuri

1-0:
1→2→0 : 11

1-4:
1→6→4 : 11

1-5:
1→5 : 7.

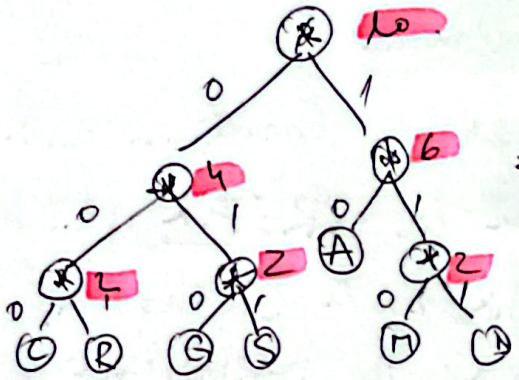
	0	[1]	[2]	3	4	5	[6]	7
D	INF	0	INF	INF	INF	INF	INF	INF
T	0	0	0	0	0	0	0	0
D	INF	0	6	9	INF	7	[3]	INF
T	0	0	1	1	0	1	[1]	0
D	11	0	[6]	9	11	7	3	INF
T	2	0	[1]	1	6	1	1	0

...

Scanned with CamScanner

MADAGASCAR - 10

M-1
A-4
D-1
G-1
S-1
C-1
R-1



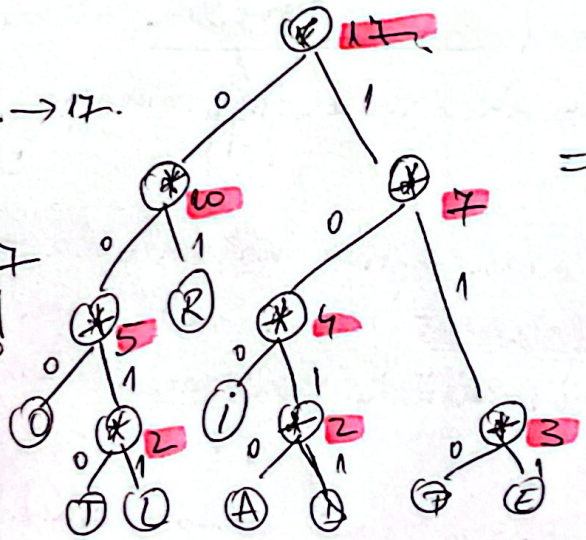
=> 26.

M: 110 → 3
A: 10 → 8
D: 111 → 3
G: 010 → 3
S: 011 → 3
C: 000 → 3
R: 001 → 3

6
20
26
6
12
6

RADIOREPORTERILOR → 17.

R-5
A-1
D-1
i-2
O-3
E-2
P-1
T-1
L-1



=> 50

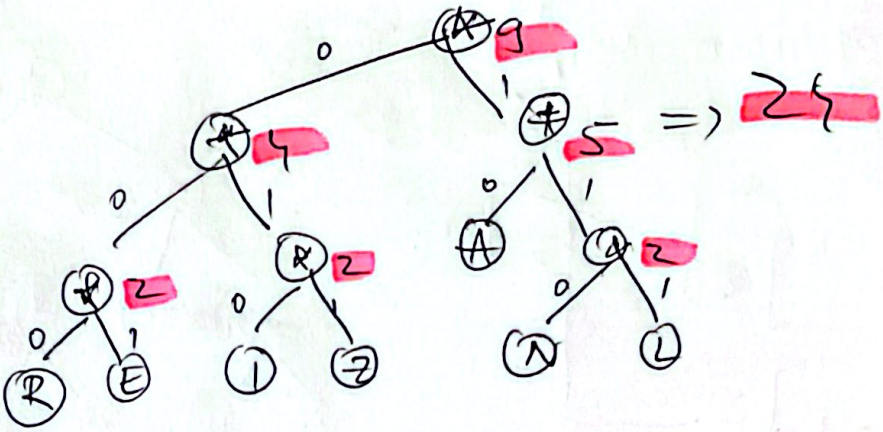
R: 01 → 10
A: 1010 → 4
D: 1011 → 4
i: 100 → 6
O: 000 → 9
E: 111 → 6
P: 110 → 3
T: 0010 → 4
L: 0011 → 4

14
10
24
15
7
11
26
50.

Scanned with CamScanner

ANALIZARE - 9

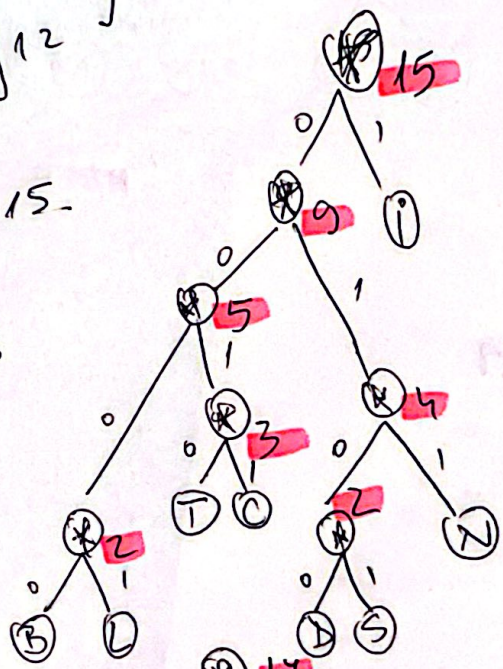
A - 3
N - 1] 2] 5
L - 1
i - 1]
Z - 1] 2] 4] 9
R - 1
E - 1] 2



A : 10 → 6
N : 110 → 3] 6] 12
L : 111 → 3
i : 010 → 3]
Z : 011 → 3] 6] 12] 24.
R : 000 → 3]
E : 001 → 3] 6

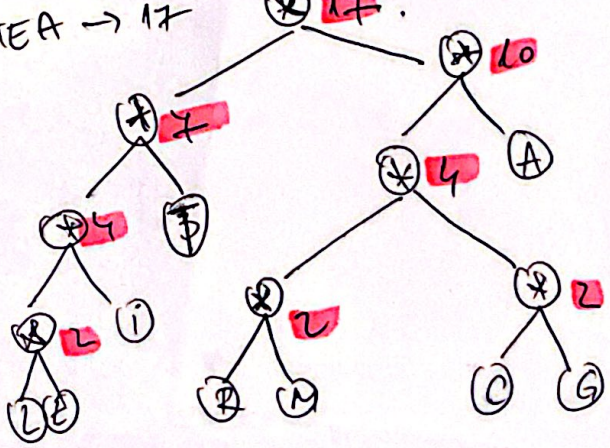
INDISTINCTIBILI → 15.

i - 6
N - 2
Δ - 1
S - 1] 2] 4
T - 2
C - 1] 3] 9] 15
B - 1
L - 1] 2



AGRAMATICALITATEA → 17

A - 6
G - 1] 4
R - 1
M - 1] 2] 2
T - 3
i - 2]
C - 1]
L - 1] 2] 4
E - 1

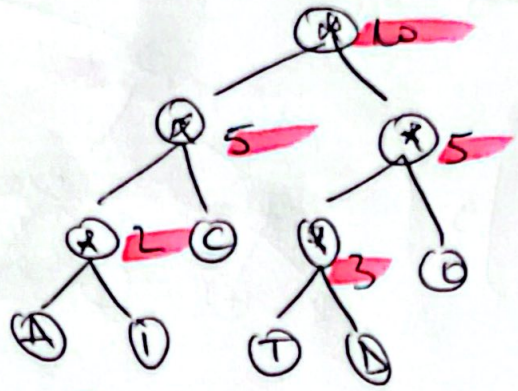


Scanned with CamScanner

COTCODACIT - 10

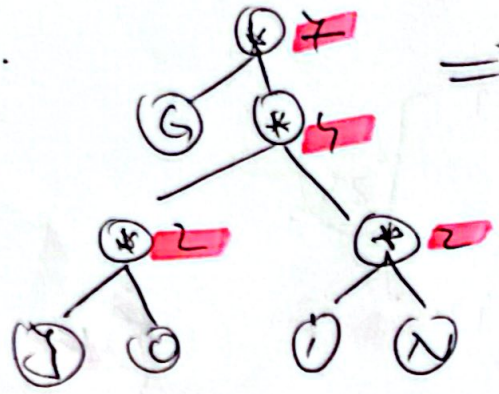
C - 3
O - 2
T - 2] 3] 5
D - 1
A - 1] 2
i - 1

5 5 10.



JOGGING - 7.

J - 1] 2
O - 1
G - 3] 4 7
I - 1
N - 1] 2



=> ~~25.~~

=> ~~15~~

Extra:
VEVERIȚĂ => 19

Scanned with CamScanner